

Revenue Captain

Blue Lake — Packet II Technical Walkthrough

Girdhari Lal | girdharilal.aiml@gmail.com | github.com/Girdharilal-aiml

1. What the project does

Revenue Captain is a hiring workflow automation service. It reads three JSON files — applications, engagement events, and school routes — normalizes them into a clean state model, detects problems in each record, and produces a prioritized action queue so the hiring captain always knows what to do next.

The pipeline runs in four steps:

Step	Function	What it does
1	normalize()	Reads raw data, builds one clean record per applicant
2	detect_issues()	Flags duplicates, bounces, missing evidence, stale records
3	build_action_queue()	Converts flags into ranked next actions
4	run_worker()	Orchestrates all steps, saves state idempotently

2. Input files

applications.json

Each record is one applicant. Key fields: id, name, email, phone, school_route_id, submitted_at, evidence.

```
{
  "id": "app_001",
  "name": "Sara Khan",
  "email": "sara@gmail.com",
  "submitted_at": "2026-06-01T09:00:00Z",
  "evidence": { "resume": true, "github": "https://...", "cover_note": true }
}
```

events.json

Activity logs per applicant — email sent, opened, replied, bounced. Used to track engagement and detect bounces.

```
{ "app_id": "app_002", "type": "email_bounced", "timestamp": "2026-05-10T15:01:00Z"
}
```

school_routes.json

School pipeline contacts. If active=false, the route is broken and outreach won't land.

```
{ "id": "route_B", "school": "IBA Karachi", "active": false, "bounce_reason": "contact no longer valid" }
```

3. Key design decisions

Fingerprinting for duplicate detection

Each applicant gets a fingerprint — a SHA-256 hash of their email + phone. If two records share the same fingerprint, they're the same person. The earliest submission is kept as primary, the rest are skipped from the queue (but kept in state with a `duplicate_of` reference so nothing is lost).

```
key = f"{email.lower()}|{phone}"
fingerprint = sha256(key).hexdigest()[:16]
```

Why SHA-256: stable, deterministic, avoids raw PII comparisons in logic.

Weighted evidence scoring

Not all evidence is equal. A resume is critical — you can't evaluate someone without it. GitHub and cover note are supporting signals.

Field	Weight	Reasoning
resume	2	Critical — can't proceed without it
github	1	Supporting signal
cover_note	1	Supporting signal
Passing threshold	3	Resume + at least one other

Flag priority suppression

Flags are applied in priority order. Higher-priority issues suppress lower ones — no point flagging 'stale' when a route is broken and you can't reach the person anyway.

```
duplicate → route_inactive / email_bounced → missing_evidence → stale
```

Example: Ahmed Raza has a bounced email. He does NOT also get a stale flag. Fix the email first — then reassess.

Route inactive vs email bounce — different problems

These look similar but need different actions:

Problem	Action	Reason code	Why
School route inactive	human_review	SCHOOL_ROUTE_INACTIVE	School-side fix, needs manual update
Applicant email bounced	fix_bounced_route	EMAIL_BOUNCED	Candidate-side, can retry or find alt contact

4. Action queue — full priority table

Priority	Action	Reason code	When triggered
1	human_review	DUPLICATE_APPLICATION	Primary of a duplicate group
1	human_review	SCHOOL_ROUTE_INACTIVE	Route marked inactive
2	fix_bounced_route	EMAIL_BOUNCED	Email bounced, route is fine
3	request_evidence	EVIDENCE_INCOMPLETE	Score below 3, lists exact missing fields
4	follow_up	NO_RECENT_ACTIVITY	Idle 14+ days, no other issues
5	advance	READY	Full evidence, active route, recent engagement

5. Idempotency — why it matters

The worker can run 100 times on the same data and the result will always be the same. No duplicate records, no corrupted state.

How it works:

```
if state_path.exists():
    old_state = load_json(state_path)
    for app_id, old_rec in old_state.items():
        if app_id in new_state:
            # These never change after first run
            new_state[app_id]["submitted_at"] = old_rec["submitted_at"]
            new_state[app_id]["fingerprint"] = old_rec["fingerprint"]
```

submitted_at and fingerprint are frozen after the first run. Flags recompute fresh from source data every run — so fixing a route or adding evidence immediately reflects on next run without touching the stable fields.

6. Complexity

Function	Complexity	Why
normalize()	O(n)	One pass through applications + events
detect_issues()	O(n)	One pass through state dict
build_action_queue()	O(n log n)	Sort at the end
Overall	O(n log n)	Dominated by sort

7. Tests — 27/27 passing

Category	Tests
Fingerprint	Same identity = same hash, different email = different hash
Evidence scoring	Full (4/4), partial, empty, exact missing fields listed
Duplicates	Secondary flagged, excluded from queue, primary kept with reference
Stale	Flagged after 14 days, suppressed by bounce, saved by recent event
Missing evidence	Flagged with specifics, suppressed when route is bounced
Bounced route	Route inactive vs email bounce handled separately
Action queue	Priority ordering, days_inactive present, engagement signals
Idempotency	3 reruns — identical state, identical queue order

8. Easy modifications for the live screen

If they ask you to change something during the oral, here are common ones:

Change stale threshold

```
STALE_DAYS = 30 # was 14
```

Add LinkedIn to evidence

```
EVIDENCE_WEIGHTS = {  
    "resume": 2,  
    "github": 1,  
    "cover_note": 1,  
    "linkedin": 1 # new  
}
```

Raise evidence requirement

```
EVIDENCE_MIN_PASS = 4 # was 3
```

Add a new urgent action

```
PRIORITY["urgent_review"] = 0 # beats human_review
```

GitHub: github.com/Girdharilal-aiml/blue-lake-revenue-captain | Tests: `python -m pytest tests/ -v` | Run: `python service.py`